

Design Principles

A design principle is a default, not a rule. Each one describes what usually goes wrong when it is ignored, and each one causes its own damage when applied without judgment.

A reading order:

SOLID → KISS → DRY → YAGNI → Composition over inheritance
→ Coupling and cohesion → Four more → When they conflict

1. SOLID

SOLID: A set of five principles for writing maintainable and flexible object-oriented code.



The first two are about placing code. The last three are about dependencies between types.

S, Single Responsibility Principle

A class should have one reason to change.

```
// three reasons to change: the calculation, the format, the storage
class Payslip {
    BigDecimal calculateNet() { }
    String toPdf() { }
    void save() { }
}
```

A new tax rule, a new document layout and a new database each force an edit to the same class, and each edit risks the other two. Splitting it into `PayslipCalculator`, `PayslipRenderer` and `PayslipRepository` gives each change one place to land.

The common mistake is reading it as "a class should do one thing" and producing fifty classes with one method each. The unit is a *reason to change*, which usually means one group of people asking for it.

O, Open Closed Principle

Open for extension, closed for modification. Adding a case should not mean editing code that already works.

```
// every new payment type edits this method
BigDecimal fee(String type, BigDecimal amount) {
    if (type.equals("CARD")) return amount.multiply(new BigDecimal("0.02"));
    if (type.equals("UPI")) return BigDecimal.ZERO;
    ...
}
```

```
// a new payment type is a new class
public interface PaymentMethod {
    BigDecimal fee(BigDecimal amount);
}
```

The `switch` is not always wrong. It is wrong when the list grows often, because every addition risks breaking the cases already there.

L, Liskov Substitution Principle

A subtype must work anywhere its parent is expected.

```
class Rectangle {
    void setWidth(int w) { this.width = w; }
    void setHeight(int h) { this.height = h; }
}

class Square extends Rectangle {
    @Override void setWidth(int w) { this.width = w; this.height = w; }
    @Override void setHeight(int h) { this.width = h; this.height = h; }
}
```

```
void resize(Rectangle r) {
    r.setWidth(5);
    r.setHeight(4);
    assert r.area() == 20; // fails for a Square
}
```

A square is a rectangle in mathematics and not in code, because the setters make a promise the square cannot keep.

The JDK has the same shape. `Arrays.asList(...)` returns a `List` whose `add` throws `UnsupportedOperationException`, so it is not usable everywhere a `List` is expected.

The signal is an override that throws, or that ignores its argument.

I, Interface Segregation Principle

No class should be forced to implement methods it does not use.

```
interface Worker {
    void work();
    void eat();
}

class Robot implements Worker {
    public void work() { }
    public void eat() { throw new UnsupportedOperationException(); }
}
```

Two interfaces, `Workable` and `Feedable`, let each class implement only what it does. A class can implement many interfaces, so splitting costs nothing.

D, Dependency Inversion Principle

High-level code should depend on an abstraction, not on a detail.

```
// the service is tied to one implementation
class OrderService {
    private final SmtplibEmailSender sender = new SmtplibEmailSender();
}

// the service states what it needs
class OrderService {
    private final NotificationSender sender;

    OrderService(NotificationSender sender) {
        this.sender = sender;
    }
}
```

The interface belongs to the caller, not the implementer. `OrderService` defines what it needs from a notification sender, and the email, SMS and test implementations fit that shape.

Dependency injection is the mechanism. Dependency inversion is the principle, and the two are often confused because Spring supplies the first.

2. KISS

KISS (Keep It Simple, Stupid): Keep code and design as simple as possible.

A discount of ten percent, built for a requirement nobody asked for:

```

public interface DiscountStrategy {
    BigDecimal apply(BigDecimal amount);
}

public class DiscountEngine {

    private final Map<String, DiscountStrategy> strategies = new HashMap<>();

    public void register(String name, DiscountStrategy strategy) {
        strategies.put(name, strategy);
    }

    public BigDecimal apply(String name, BigDecimal amount) {
        return strategies.getOrDefault(name, a -> a).apply(amount);
    }
}

```

The same discount:

```

public BigDecimal applyDiscount(BigDecimal amount) {
    return amount.multiply(new BigDecimal("0.90"));
}

```

The first version is not wrong. It is an answer to a question nobody asked, and it has to be read, tested and maintained until somebody does.

The test is whether the next person can change it safely. Code that needs explaining before it can be edited has already failed, however clever it is.

3. DRY

DRY (Don't Repeat Yourself): Avoids repeating the same code or logic.

```

// three places that all know the tax rate
BigDecimal invoiceTotal = subtotal.multiply(new BigDecimal("1.18"));
BigDecimal quoteTotal   = subtotal.multiply(new BigDecimal("1.18"));
BigDecimal reportTotal  = subtotal.multiply(new BigDecimal("1.18"));

```

When the rate changes, three places have to change and one of them is missed.

DRY is about knowledge, not about text. Two methods that read identically are not duplication if they change for different reasons:

```
BigDecimal invoiceTotal(List<Line> lines) { /* same body */ }
BigDecimal payrollTotal(List<Line> lines) { /* same body */ }
```

Merging those creates one method that two callers now constrain. The first time invoices need a rounding rule that payroll does not, the shared method grows a flag, then a second flag, and the abstraction is worse than the duplication ever was.

Same text, different reason to change	→ not duplication
Same knowledge, written twice	→ duplication

Two occurrences is a coincidence. Waiting for the third makes the shape of the real abstraction visible, and a wrong abstraction costs more to remove than duplication costs to keep.

4. YAGNI

YAGNI (You Aren't Gonna Need It): Don't build something until it is actually needed.

```
// "in case we move off the database one day"
public interface EmployeeStore { }

public class JpaEmployeeStore implements EmployeeStore { }
```

Six years later there is still one implementation. The interface has been read by everyone, navigated through by everyone, and has never once been swapped.

Speculative code is not free. It is tested, maintained, read, and usually wrong when the requirement finally arrives, because it was designed by guessing.

The exception is anything expensive to reverse. The database, the shape of a public API, how data is encrypted, where a boundary sits. YAGNI applies to code, not to decisions that are hard to undo once other people depend on them.

5. Composition over inheritance

Inheritance is the strongest coupling a language offers. A subclass depends on how its parent is written, not just on what it promises, and it inherits the parent's entire public surface.

```
// every Vector method comes along, including the ones that break a stack
class HistoryStack<E> extends Vector<E> { }

// exactly four operations, and the implementation can change
final class HistoryStack<E> {
    private final Deque<E> items = new ArrayDeque<>();

    void push(E item) { items.addLast(item); }
    E pop()           { return items.pollLast(); }
    E peek()          { return items.peekLast(); }
    boolean isEmpty() { return items.isEmpty(); }
}
```

Use inheritance when the relationship is genuinely "is a", the parent was designed for extension, and the parent is ours or stable. Otherwise hold the object as a field.

6. Coupling and cohesion

These two describe whether a design will be easy to change.

	Meaning	We want
Coupling	How much one part depends on another	Low
Cohesion	How closely the members of one part belong together	High

A class that names a concrete type, reaches into another object's fields, or depends on the order its methods are called is tightly coupled. A class whose methods all work on the same state is cohesive; a class named `Utils` usually is not.

Information hiding is how coupling is kept low. Every field that stays private and every method that stays package-private is a thing no other code can depend on, so it can change freely later.

7. Four more worth knowing

Principle	What it says
Law of Demeter	Talk to immediate neighbors only
Tell, Don't Ask	Tell an object what to do rather than asking for its state
Fail Fast	Reject bad input where it enters, not three layers in
POLA, Principle of Least Astonishment	A method should do what its name suggests

The first two are the same idea from two angles:

```
// asking: the caller reaches through three objects and decides
if (order.getCustomer().getAddress().getCountry().equals("IN")) {
    total = total.multiply(new BigDecimal("1.18"));
}

// telling: the object that owns the data makes the decision
total = order.applyTax(total);
```

The first version breaks when any of those three classes changes. The second couples the caller to `Order` and nothing else.

Fail Fast is what keeps a bad value from traveling:

```
public Money(BigDecimal amount) {
    if (amount.signum() < 0) {
        throw new IllegalArgumentException("amount cannot be negative");
    }
    this.amount = amount;
}
```

Checking at construction means no negative `Money` can exist anywhere. Checking at the point of use means checking in every method that uses it, and forgetting one.

Defensive programming is the same instinct applied badly when it becomes a null check on every parameter of every method. Validate at the boundary, then trust the types inside it.

POLA is the least discussed of these. A method called `getUser` that also creates a user when none exists is astonishing, and every caller who did not read the body is now wrong.

8. When they conflict

They contradict each other regularly, which is why they are defaults and not rules.

Tension	How it shows
DRY against KISS	Removing duplication adds indirection, and three hops to read one behavior
YAGNI against Open Closed	One says do not build the extension point, the other says leave room for it
Single Responsibility against simplicity	Splitting too far gives fifty one-method classes and no obvious place to start reading
Fail Fast against tolerance	An input a person typed is often better corrected than rejected

The resolution is always the same two questions. Which change is actually likely, and how expensive is it to make later? An extension point for something that will never happen is waste; one for something certain to happen next quarter is cheap now and expensive then.

Likely and expensive later → build for it now
Unlikely or cheap later → leave it simple

The pieces together

Principle	In one line
SRP	One reason to change
OCP	Extend without editing what already works
LSP	A subtype must work where its parent is expected
ISP	Small interfaces, not large ones
DIP	Depend on an abstraction, not on a detail
KISS	Keep code and design as simple as possible
DRY	One piece of knowledge in one place
YAGNI	Do not build something until it is actually needed
Composition over inheritance	Hold the object rather than extending it
Low coupling, high cohesion	Parts that can change separately
Law of Demeter	Talk to immediate neighbors only
Fail Fast	Reject bad input where it enters

An order service, for example. `Order` validates its own totals in its constructor, so no invalid order exists anywhere. The tax rate lives in one class, not in three. `PaymentMethod` is an interface, so a new provider is a new class rather than another branch in a `switch`. There is no `OrderStore` interface, because there is one database and no plan to change it. The service takes a `NotificationSender` in its constructor and does not know whether it sends email.

Take one class that is hard to change, and name which principle it breaks. Usually only one of them explains the pain.